

Department of Computer Science and Engineering

Indian Institute of Technology Guwahati

Graded Lab 3

Fast Kernel-to-User Information Page

Reading process information without a system call

CS3106L - Operating Systems: Concepts and Construction

BrahmaputraOS / Luit - Student Release v1.2

At a glance

Marks: 100. Grading is fully automated. No viva and no report are required.

You will complete four TODO regions in two source files. The page-table mapping and process-lifecycle code are already provided. Your job is to make the kernel update the page safely and to make user code read a consistent snapshot without an `ecall`.

Contents

1	What are we building?	2
2	What is already provided, and what do you write?	2
2.1	Already provided	2
2.2	You implement exactly four TODO regions	3
3	Start from a clean copy	3
4	The shared structure	4
5	Why a normal struct copy is not enough	4
5.1	The sequence-counter rule	5
6	Task 1 - K1: update the page in the kernel	5
6.1	What K1 must do	5
6.2	Why are the two barriers needed?	6
6.3	Why use push_off() and pop_off()?	6
7	Task 2 - U1: read one consistent snapshot	6
7.1	First check the arguments and version	7
7.2	Then use a retry loop	7
8	Task 3 - U2 and U3: fast user functions	7
8.1	U2: u_getpid()	7
8.2	U3: u_uptime()	8
9	Test your work	8
9.1	1. Run the simple Lab 3 check	8
9.2	2. Clean build	8
9.3	3. Run the normal regression tests	8
9.4	4. Run the Lab 3 public grader	8
9.5	5. Optional observation: compare the two paths	9
10	Automatic grading - 100 marks	9
11	Exactly what to submit	10
12	Common mistakes	11
13	Final checklist	11

1 What are we building?

In Lab 2, you worked inside the system-call path. In the non-graded practice lab, you also inspected page tables and the read-only shared page using GDB.

This graded lab uses both ideas.

Normally, a user program asks the kernel for information by making a system call:

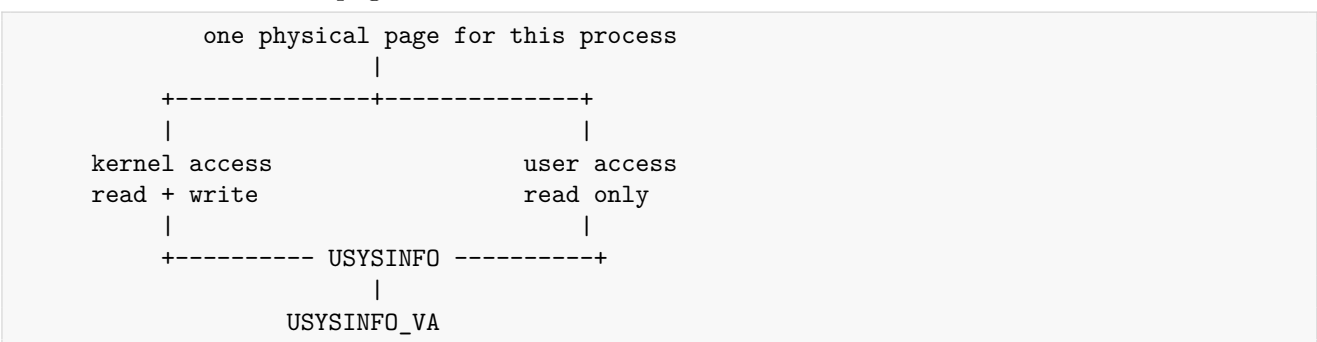
```
user program -> ecall -> trap -> syscall() -> kernel -> return to user
```

That is correct, but an `ecall` has a cost. For small values that are read very often, the kernel can instead keep a page of information mapped into the process address space.

For this lab, every process already has one page at:

```
USYSINFO_VA = 0x7FFF000
```

The kernel can write this page. User code can read it. User code cannot write it.



Your goal is to make calls such as `u_getpid()` read from this page instead of entering the kernel.

The main problem

The page contains several fields. The kernel may be updating those fields while the user is reading them. A read-only page prevents the user from changing the page, but it does **not** guarantee that several fields are read from the same kernel update.

You will solve this using a small sequence counter called `seq`.

2 What is already provided, and what do you write?

2.1 Already provided

You do **not** need to implement the following parts:

- allocation of one information page for each process;
- mapping that page at `USYSINFO_VA`;
- user permissions `PTE_R` | `PTE_U`;
- no user write permission (there is no `PTE_W`);
- handling of the page during process exit;
- remapping of the same page when `exec()` creates a new page table;
- preventing `sbrk()` from growing the normal process image into the reserved top page;
- process counters such as `syscall_count` and `ctxsw_count`;

- the places in the kernel that call `usysinfo_update()`;
- public test programs.

You should read this supplied code because the tests depend on it, but do not redesign it for this lab.

2.2 You implement exactly four TODO regions

File	TODO	What you implement
kernel/usysinfo.c	K1	Kernel safely updates all fields in the shared page.
user/ulib.c	U1	User reads one consistent snapshot and retries when necessary.
user/ulib.c	U2	Fast <code>u_getpid()</code> using the shared page.
user/ulib.c	U3	Fast <code>u_uptime()</code> using the shared page.

Do not remove the TODO markers

Keep the lines containing `TODO-BEGIN K1`, `TODO-END K1`, and the `U1/U2/U3` markers. The official grader extracts only the code between these markers and inserts it into a clean staff copy of Luit. Changes outside these four TODO regions are not used for grading. **Therefore, all code that you write for this lab must stay inside K1, U1, U2, or U3.** Do not add helper functions, new declarations, or extra `#include` lines outside the TODO regions. Such code may work in your local directory, but the official grader will discard it.

This is also why the lab does **not** use a checksum of your entire directory. The grader does not need one.

3 Start from a clean copy

Please use the Lab 3 archive provided for this assignment. Do not continue inside an old Lab 1 or Lab 2 directory.

From the Linux terminal:

```
unzip LUIT_Lab03_Student_Distribution_v1.2.zip
cd LUIT_Lab03_Student

make lab03-check
make clean
make
make grade
```

On the untouched starter, the normal Luit regression tests should pass. The Lab 3-specific test is expected to fail because `K1/U1/U2/U3` are still incomplete.

You can confirm this with:

```
make grade LAB=3
```

Before you start coding

If `make`, `make grade`, or the baseline system fails on a fresh extraction, stop there. That is an environment or release problem, not your Lab 3 solution.

4 The shared structure

Open `kernel/usysinfo.h`. The structure is supplied to you:

```
#define USYSINFO_VERSION 1
#define USYSINFO_VA      0x7FFFFF000UL

struct usysinfo {
    uint32 version;
    volatile uint32 seq;
    int pid;
    int hart;
    uint64 ticks;
    uint64 syscall_count;
    uint64 ctxsw_count;
    uint32 state_gen;
};
```

The fields mean:

Field	Meaning
<code>version</code>	Layout version. User code must check that it understands this version.
<code>seq</code>	Sequence counter used to detect an update that overlaps a user read.
<code>pid</code>	PID of the process that owns this page.
<code>hart</code>	Hart on which the most recent update was made.
<code>ticks</code>	Kernel tick value seen at the most recent update. It can be slightly older than a fresh <code>uptime()</code> syscall.
<code>syscall_count</code>	Number of completed system calls for this process.
<code>ctxsw_count</code>	Number of times the scheduler has selected this process to run.
<code>state_gen</code>	A second counter updated together with <code>ctxsw_count</code> ; the tests use it to detect mixed snapshots.

For the duration of this lab, the supplied kernel keeps the low 32 bits of `ctxsw_count` equal to `state_gen`. A correct snapshot should therefore satisfy:

```
(uint32)s.ctxsw_count == s.state_gen
```

5 Why a normal struct copy is not enough

Suppose the page currently contains an old snapshot:

```
seq = 10, ticks = 100, syscall_count = 20, ctxsw_count = 7
```

The user begins reading. In the middle of the copy, the kernel updates the page:

```
User reads:      ticks = 100
Kernel updates:  ticks = 101, syscall_count = 21, ctxsw_count = 8
User reads:      syscall_count = 21, ctxsw_count = 8
```

The user has now combined old and new values. That is a **torn snapshot**.

We need a way for the reader to detect this and try again.

5.1 The sequence-counter rule

We use this simple rule:

- **even seq**: the page is stable;
- **odd seq**: the kernel is currently changing the page.

For example:

```
seq = 10    stable
seq = 11    kernel update in progress
seq = 12    new stable snapshot
```

A user reader does this:

1. read **seq** before copying the fields;
2. if it is odd, retry;
3. copy the fields;
4. read **seq** again;
5. accept the copy only if both sequence values are equal and even;
6. otherwise, retry.

If the kernel changes the page while the user is copying it, the sequence value will change and the user will reject that copy.

6 Task 1 - K1: update the page in the kernel

Open:

```
kernel/usysinfo.c
```

Find `TODO-BEGIN K1`.

Your function is:

```
void usysinfo_update(struct proc *p)
```

6.1 What K1 must do

Implement these steps in this order:

1. Get the shared-page pointer from `p->usysinfo`. If it is zero, return.
2. Call `push_off()` before starting the update.
3. Increase **seq** by one. Since a stable value is even, it now becomes odd.
4. Call `__sync_synchronize()`.
5. Copy these fields from the process/kernel state into the shared page:
 - `pid`
 - `hart`

- `ticks`
- `syscall_count`
- `ctxsw_count`
- `state_gen`

6. Call `__sync_synchronize()` again.

7. Increase `seq` again. It should now be even.

8. Call `pop_off()`.

Do not change `version` inside `K1`. The setup code has already initialized it to `USYSINFO_VERSION`.

6.2 Why are the two barriers needed?

Compilers and CPUs are allowed to reorder memory operations when that does not change a single-threaded program.

Here, the order is part of the protocol. We need:

```
odd seq must be visible BEFORE field updates
field updates must be visible BEFORE final even seq
```

For this assignment, use `__sync_synchronize()` at the two places listed above. This keeps the implementation and the automatic checker simple.

6.3 Why use `push_off()` and `pop_off()`?

This part is specific to Luit.

Normal system calls run with interrupts enabled. A timer interrupt can therefore arrive while `usysinfo_update()` is halfway through an update. The timer path may call `yield()`, and the scheduler also calls `usysinfo_update()` when it selects a process.

Without protection, the same shared page could have two overlapping writers.

Use the short, nest-aware pair:

```
push_off();
/* update only; do not sleep here */
pop_off();
```

Do not use a raw “interrupt off, then always interrupt on” pair. `push_off()/pop_off()` correctly preserves an outer interrupt-disabled section if one already exists.

Expected result after K1

After `K1` is correct, the kernel should leave `seq` even whenever it is not updating the page. All six data fields should come from one update window. The system should still boot normally on one and several harts.

7 Task 2 - U1: read one consistent snapshot

Open:

```
user/ulib.c
```

Find TODO-BEGIN U1 inside:

```
int u_snapshot(struct usysinfo *out)
```

7.1 First check the arguments and version

Return -1 if:

- `out == 0`; or
- the page's `version` is not `USYSINFO_VERSION`.

Otherwise, read from the page at `USYSINFO_VA`.

7.2 Then use a retry loop

Your loop should follow this logic:

```
read seq -> before
if before is odd: retry

barrier
copy every field into *out
barrier

read seq -> after
if before == after and after is even: success
otherwise: retry
```

Use `__sync_synchronize()` for the two reader barriers as well.

Copy every field in `struct usysinfo`, including `version` and `seq`. When you accept the snapshot, store the final stable sequence value in `out->seq`.

Do not use a fixed number of retries. If the writer is active, the reader should simply try again.

Expected result after U1

`u_snapshot(&s)` should return 0. The returned `s.seq` should always be even. Under stress, the test should not observe a mixed snapshot, and `(uint32)s.ctxsw_count == s.state_gen` should remain true.

8 Task 3 - U2 and U3: fast user functions

Now complete the two small helpers in `user/ulib.c`.

8.1 U2: `u_getpid()`

Required behavior:

1. call `u_snapshot()`;
2. if it succeeds, return the PID from the snapshot;
3. if it returns -1, fall back to the normal `getpid()` system call.

On the normal Lab 3 page, `u_getpid()` must not execute an `ecall`.

8.2 U3: `u_uptime()`

Required behavior:

1. call `u_snapshot()`;
2. if it succeeds, return `ticks` from the snapshot;
3. if it returns `-1`, fall back to the normal `uptime()` system call.

The shared `ticks` value is updated when the kernel publishes the page. It may therefore be a little older than a fresh `uptime()` syscall. The public test allows a small difference.

Expected result after U2 and U3

`u_getpid()` should match `getpid()`. `u_uptime()` should track `uptime()` closely. Repeated calls to the fast functions should not increase the process's `syscall_count`.

9 Test your work

Use this order. It makes debugging much easier.

9.1 1. Run the simple Lab 3 check

```
make lab03-check
```

This check does **not** compare checksums. It only checks that the four TODO markers are still present so that your submission can be extracted correctly.

9.2 2. Clean build

```
make clean
make
```

Fix all compiler warnings and errors. The course build uses `-Werror`.

9.3 3. Run the normal regression tests

```
make grade
```

Your Lab 3 changes must not break the existing Luit system.

9.4 4. Run the Lab 3 public grader

```
make grade LAB=3
```

The public Lab 3 test runs at both `CPUS=1` and `CPUS=4`.

A correct solution should end with public Lab 3 passes in both configurations.

The public test checks, among other things:

- null output is rejected;

- the ABI version is correct;
- `u_snapshot()` returns the correct PID;
- returned sequence values are stable and even;
- `ctxsw_count` and `state_gen` are from the same snapshot;
- `u_getpid()` does not execute a syscall on the fast path;
- `u_uptime()` does not execute a syscall on the fast path;
- real system calls still increase `syscall_count`;
- 100,000 repeated snapshots show no torn read;
- the shared page remains user-read-only;
- different processes use different information pages;
- the page still works after `exec()`.

9.5 5. Optional observation: compare the two paths

You may run:

```
make qemu
```

and then, at the `luit$` prompt:

```
usibench 500000
```

This is for observation only. Timing is noisy and is **not** used for marks. The stronger proof that the fast path avoided `ecall` is the `syscall_count` check in the tests.

10 Automatic grading - 100 marks

There is no viva and no report. All marks come from automatic checks on a clean staff copy of Luit.

Automatic check	Marks	What is tested
Submission format	5	Correct ZIP shape and all four TODO markers can be extracted.
Clean build	10	Your extracted TODO code compiles in the staff Lab 3 checkpoint.
Existing Luit regression	10	Baseline OS tests still pass after your TODO code is inserted.
Basic shared-page behavior	20	Version, null handling, fields, PID, sequence state, and snapshot interface.
Fast-path behavior	15	<code>u_getpid()</code> and <code>u_uptime()</code> use the page without a normal-path <code>ecall</code> ; real syscalls are still counted.
Consistency stress	15	Repeated snapshots on one hart remain consistent and satisfy the cross-field invariant.
SMP and integration	10	Lab 3 tests on multiple harts plus read-only protection, process isolation, and <code>exec()</code> behavior.
Long hidden stress	5	Longer repeated-update/snapshot run.
Version-fallback hidden test	5	Fast helpers fall back safely when the shared-page ABI version is deliberately made incompatible.
Required implementation operations	5	Automatic source check for the listed K1 field assignments, <code>push_off/pop_off</code> , and the required <code>__sync_synchronize()</code> operations.
Total	100	

The hidden tests follow the same requirements described in this handout. They do not require an undocumented interface.

11 Exactly what to submit

You submit only one ZIP file.

From the root of your Lab 3 directory, run:

```
python3 tools/package_lab03.py YOUR_ROLL_NUMBER
```

For example:

```
python3 tools/package_lab03.py 260101001
```

The script creates:

```
260101001_Lab03.zip
```

The ZIP is checked immediately after it is created. It must contain **exactly** these two files:

```
260101001_Lab03/kernel/usysinfo.c
260101001_Lab03/user/ulib.c
```

No PDF, report, screenshot, binary, full source tree, checksum file, or test output is required.

Important submission rule

Upload the ZIP created by `package_lab03.py`. Do not open it and add extra files manually. During grading, the staff script reads K1/U1/U2/U3 from these two files and inserts only those TODO bodies into a pristine Lab 3 checkpoint. This avoids the whole-tree checksum and validation problems that can happen when different machines create or modify unrelated files.

Before uploading, run one final time:

```
make lab03-check
make clean && make
make grade
make grade LAB=3
python3 tools/package_lab03.py YOUR_ROLL_NUMBER
```

12 Common mistakes

Symptom / mistake	What to check
<code>u_snapshot</code> always returns -1	Did you leave the starter U1 return in place? Did you check the correct version?
<code>seq</code> is sometimes odd in returned snapshots	The reader must reject an odd starting value and must read <code>seq</code> again after copying.
<code>ctxsw_count != state_gen</code>	A torn snapshot was accepted, or the writer did not keep all field stores inside the sequence window.
Public test hangs	Check that the writer always ends with even <code>seq</code> and that <code>push_off/pop_off</code> are balanced.
Fast <code>getpid</code> changes <code>syscall_count</code>	U2 is calling <code>getpid</code> on the normal path instead of returning the shared-page PID.
Fast <code>uptime</code> changes <code>syscall_count</code>	U3 is calling <code>uptime</code> on the normal path instead of returning the shared-page tick value.
Works at <code>CPUS=1</code> but fails at <code>CPUS=4</code>	Check barriers and the short writer critical section.
Kernel hangs after a timer interrupt	Check that <code>push_off/pop_off</code> are balanced; do not sleep inside the K1 critical section.
<code>package_lab03.py</code> fails	Keep exactly one begin/end marker for K1/U1/U2/U3 and run the script from the Lab 3 root.

13 Final checklist

Before you submit, make sure all of the following are true:

- K1 updates `pid`, `hart`, `ticks`, `syscall_count`, `ctxsw_count`, and `state_gen`.
- K1 uses odd/even `seq` correctly.
- K1 uses `push_off()` and `pop_off()`.
- K1 uses two `__sync_synchronize()` barriers.
- U1 checks out and `USYSINFO_VERSION`.
- U1 reads `seq` before and after the copy and retries unless both values are equal and even.

- U1 uses two `__sync_synchronize()` barriers.
- U1 copies every field in `struct usysinfo`.
- U2 returns the shared PID on the normal path.
- U3 returns the shared tick value on the normal path.
- U2/U3 use system calls only as the version/error fallback.
- `make grade` passes.
- `make grade LAB=3` passes at the end.
- Your final ZIP was created by `package_lab03.py` and contains only the two required source files.

The key idea: permission protects the page from user writes; the sequence protocol protects the reader from a half-updated snapshot.